SPEC-DRIVEN DEVELOPMENT

TaskAnalyzer

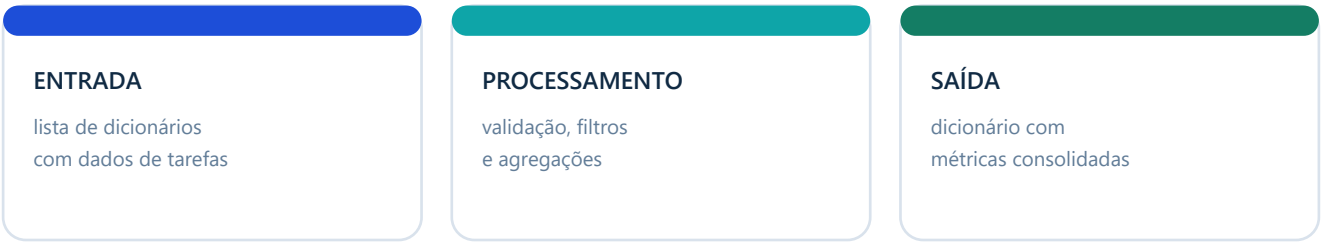
Documento de Design do Sistema

ESCOPO	BASE VALIDADA	EDIÇÃO
Módulo de análise de tarefas	branch main commit b69a1dc	1.0 23 out 2026

Documento descritivo. O contrato normativo permanece em specs/task_analyzer_spec.md; specs/requirements.yaml permanece sua transposição estruturada.

1. Visão executiva

O TaskAnalyzer é um módulo Python que recebe uma lista de tarefas, valida os dados definidos no contrato SDD e devolve métricas de produtividade. O sistema é deliberadamente pequeno: não possui persistência, interface gráfica, API HTTP nem etapa de implantação. Seu foco é um contrato público estável, cálculo determinístico e validação automatizada.



Objetivo

Gerar totais, médias de conclusão, taxa de atraso e métricas por prioridade a partir dos dados recebidos, observando os casos de borda e as restrições do contrato.

Escopo documentado

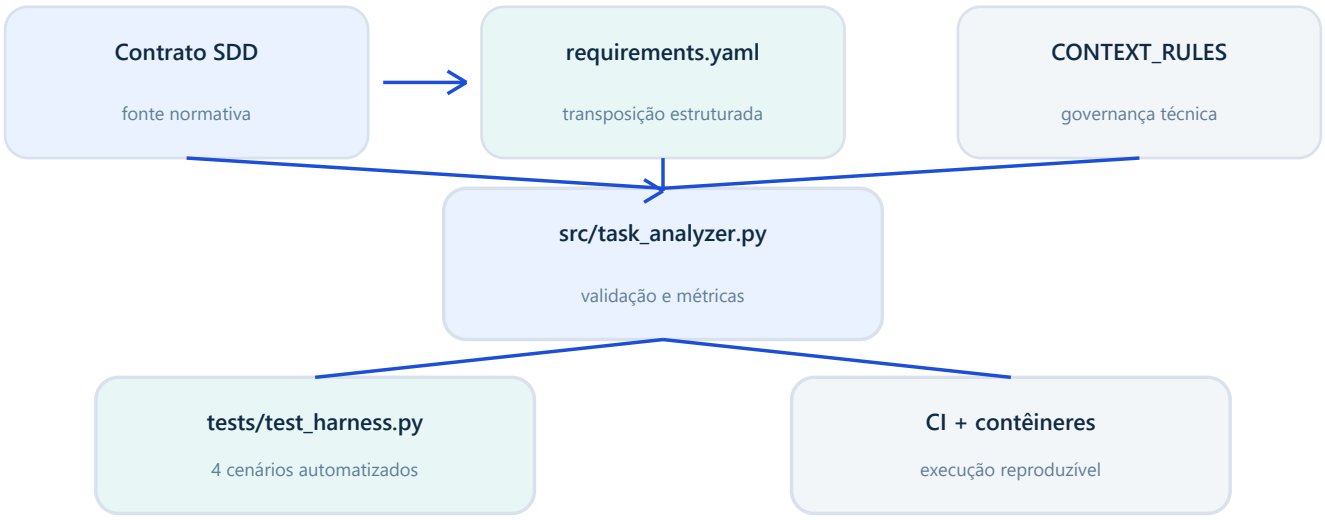
Incluído	Fora do escopo atual
Interface pública TaskValidationError e analyze_tasks	Persistência em arquivos ou banco de dados
Validação dos dados previstos	Serviço web, tela ou integração externa
Cálculo de métricas gerais e por prioridade	Implantação automatizada em ambiente de produção
Test Harness, CI e execução por contêiner	Comportamentos não definidos pelo contrato

Princípios

- O contrato precede a implementação e limita seu comportamento.
- A automação fornece evidência; a decisão de homologação permanece humana.
- Simplicidade, legibilidade e ausência de estado persistido reduzem o acoplamento.

2. Fontes de verdade e governança

O projeto separa três papéis documentais. Essa separação evita que uma representação derivada substitua silenciosamente o contrato original.



Artefato	Papel	Relação de autoridade
specs/task_analyzer_spec.md	Contrato SDD original: interface, dados, métricas, regras, cenários e restrições.	Fonte normativa de negócio.
specs/requirements.yaml	Representação estruturada do mesmo conteúdo para leitura mecânica e rastreabilidade.	Derivada; não substitui nem amplia o contrato.
CONTEXT_RULES.md	Diretrizes técnicas, proibições e regras para uso de IA.	Governa como o contrato é implementado e revisado.
system_design_document.pdf	Descrição consolidada do desenho e dos mecanismos já presentes.	Informativo; não cria requisitos de negócio.

Restrições arquiteturais vigentes

- Python 3.11 ou superior, type hints e estilo PEP 8.
- Funções pequenas, coesas, legíveis e com responsabilidade única; Google Style Docstrings.
- Exceções específicas, prevenção de operações inválidas e ausência de dependências não autorizadas.
- Sem persistência, alteração de APIs públicas ou comportamentos fora do contrato.

Regra de preservação: qualquer divergência futura deve ser resolvida no contrato SDD original e homologada por uma pessoa antes de atualizar representações derivadas ou implementação.

3. Contrato funcional

Interface pública

```
class TaskValidationError(Exception): ...

def analyze_tasks(tasks: list[dict[str, object]]) -> dict[str, object]: ...
```

Dados de entrada

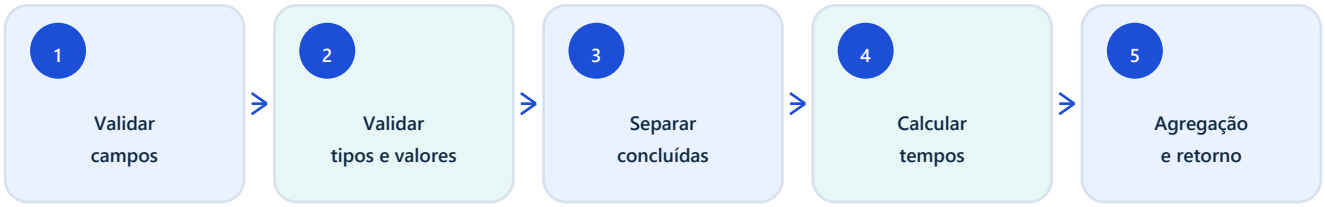
Campo	Tipo / domínio	Observação do contrato
id_tarefa	int	Inteiro positivo.
data_criacao	datetime	Obrigatória e comparável.
data_inicio	datetime | None	Disponível quando aplicável.
data_conclusao	datetime | None	Não pode ser anterior à criação.
prazo	datetime	Referência para atraso.
prioridade	baixa | media | alta	Domínio fechado.
status	concluida | pendente | cancelada	Domínio fechado.

Saída mínima

Chave	Semântica
total_tarefas	Quantidade total de itens recebidos.
total_concluidas	Itens com status concluida.
total_pendentes	Itens com status pendente.
tempo_medio_conclusao_horas	Média geral, em horas, para tarefas concluídas elegíveis.
taxa_atraso_percentual	Percentual de concluídas cuja conclusão ocorreu após o prazo.
por_prioridade	Agrupamento com média de conclusão por prioridade.

4. Implementação

A implementação está concentrada em src/task_analyzer.py. Ela mantém a API pública pequena e distribui o trabalho entre validação de datas, validação de tarefas, cálculo seguro de médias e agregação principal.



Componentes internos

Elemento	Responsabilidade observada	Falha / retorno
PRIORIDADES_VALIDAS / STATUS_VALIDOS	Definem os valores aceitos pela implementação.	Base para validação de domínio.
_validar_data	Aceita datetime e, quando configurado, None.	TaskValidationError para tipo inválido.
_validar_tarefa	Verifica campos, identificador, domínio, datas e dados de tarefas concluídas.	Interrompe a análise com erro específico.
_media	Calcula média arredondada a duas casas.	Retorna 0.0 para lista vazia.
analyze_tasks	Valida, filtra, calcula tempos e atraso, agrupa e monta o retorno.	dict[str, object] conforme contrato.

Fluxo de execução

- Cada tarefa é validada antes de qualquer agregação.
- Somente tarefas com status concluida participam dos tempos e da taxa de atraso.
- O tempo usa data_conclusao - data_inicio, convertido de segundos para horas.
- Uma concluída é atrasada quando data_conclusao > prazo.
- As prioridades baixa, media e alta aparecem no agrupamento; grupos sem valores recebem 0.0.

A implementação é sem estado: a função trabalha apenas com o argumento recebido e não lê nem grava arquivos ou banco de dados.

5. Métricas e comportamento de borda

Definições operacionais

Métrica	Cálculo implementado	Proteção
Total	len(tasks)	Lista vazia produz 0.
Concluídas	Quantidade com status concluida	Demais status não entram.
Pendentes	Soma de comparações status == pendente	Resultado inteiro.
Tempo médio	sum(tempos) / len(tempos), arredondado a 2 casas	0.0 sem valores.
Taxa de atraso	atrasadas / total_concluidas x 100, arredondado a 2 casas	0.0 sem concluídas.
Por prioridade	Média dos tempos em cada prioridade válida	0.0 para grupo vazio.

Erros previstos e tratamento

Condição validada	Comportamento implementado
Campo obrigatório ausente	TaskValidationError com relação dos campos ausentes.
id_tarefa não inteiro ou não positivo	TaskValidationError.
Prioridade ou status fora do domínio	TaskValidationError.
Campo de data com tipo inválido	TaskValidationError identificando o campo.
Conclusão anterior à criação	TaskValidationError exigido pelo contrato.
Tarefa concluída sem início ou conclusão	TaskValidationError.

Casos de borda

- Entrada vazia: totais zerados, médias 0.0 e taxa 0.0.
- Somente pendentes: nenhuma divisão é executada com denominador zero.
- Prioridade sem tarefa concluída: média 0.0 no respectivo grupo.

6. Test Harness e rastreabilidade

O arquivo tests/test_harness.py importa a API pública e contém quatro testes pytest. Os cenários reproduzem a aceitação e os casos de borda definidos no contrato.

Teste	Evidência exercitada	Resultado esperado
test_analyze_tasks_sucesso	3 tarefas: 2 concluídas, 1 pendente; prioridades distintas e 1 atraso.	Média 3.0 h; atraso 50.0%; alta 2.0 h; baixa 4.0 h; media 0.0 h.
test_analyze_tasks_data_inconsistente	data_conclusao anterior a data_criacao.	TaskValidationError.
test_analyze_tasks_lista_vazia	Entrada sem itens.	Totais 0, média 0.0 e atraso 0.0.
test_analyze_tasks_apenas_pendentes	1 tarefa pendente, sem datas de início e conclusão.	Total 1, pendentes 1, média 0.0 e atraso 0.0.

Matriz contrato - teste

Regra / cenário	Sucesso	Data inválida	Lista vazia	Pendentes
Totais	•		•	•
Média geral	•		•	•
Taxa de atraso	•		•	•
Média por prioridade	•			
TaskValidationError		•		
Prevenção de divisão por zero			•	•

Comando canônico

```
python -m pytest -v tests/test_harness.py
```

O README registra a última validação manual no Google Colab com Python 3.13.16 e resultado de 4 testes aprovados. A automação do repositório executa a mesma suíte com Python 3.11.

7. CI/CD, Docker e Docker Compose

Integração contínua

O workflow do GitHub Actions em .github/workflows/ci.yml automatiza a validação. Ele é acionado por push e pull request direcionados às branches main e develop, com permissão de conteúdo somente para leitura.

Etapa	Configuração validada	Finalidade
Runner	ubuntu-latest	Ambiente do job de testes.
Checkout	actions/checkout@v4	Disponibiliza o código.
Python	actions/setup-python@v5; 3.11	Fixa a versão mínima do contrato.
Dependências	pip install -r requirements.txt	Instala pytest, única dependência declarada.
Validação	python -m pytest -v tests/test_harness.py	Executa os quatro cenários.

Contêiner de testes

Dockerfile	docker-compose.yml
Base python:3.11-slim e diretório /app.	Serviço único chamado tests.
Instala requirements.txt sem cache.	Build usa o Dockerfile na raiz.
Copia somente src/ e tests/.	Comando executa o mesmo Test Harness.
CMD: pytest em modo verboso.	Não define serviços de persistência ou rede adicional.

Leitura correta de CI/CD

O repositório contém integração contínua para testes, mas não contém uma etapa de entrega ou implantação. Portanto, o termo CI/CD aqui descreve a área de automação do projeto; a capacidade implementada e validada é CI.

Execução reproduzível

```
docker compose up --build --abort-on-container-exit
```

8. Validação humana e uso de IA

Assistentes de IA apoiam a elaboração e a implementação, mas não alteram a autoridade do contrato nem substituem a revisão humana. O fluxo abaixo consolida as regras de interação e o processo de versionamento descritos no projeto.

1. Contexto	2. Produção assistida	3. Evidência	4. Decisão
Fornecer contrato e regras relevantes; confirmar entendimento.	IA propõe código ou documentação dentro do escopo aprovado.	Test Harness, CI e revisão verificam aderência.	Uma pessoa corrige, rejeita ou homologa antes da integração.

Controles de governança

- Solicitar explicação quando uma decisão técnica ou comportamento não estiver claro.
- Revisar criticamente todo código produzido por IA.
- Rejeitar ou corrigir respostas que violem o contrato, as CONTEXT_RULES ou os requisitos vigentes.
- Manter a homologação final sob responsabilidade humana.

Estrutura final do repositório

```
sdd-task-analyzer/
├── .github/workflows/ci.yml
├── specs/
│   ├── task_analyzer_spec.md   # contrato SDD original
│   ├── requirements.yaml      # transposição estruturada
│   └── system_design_document.pdf
├── src/task_analyzer.py
├── tests/test_harness.py
├── CONTEXT_RULES.md
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── README.md
└── .gitignore
```

O PDF ocupa a camada documental. Ele referencia os artefatos existentes sem substituir o contrato nem a representação YAML.

9. Rastreabilidade e referências

Mapa de rastreabilidade

Tema	Definição	Realização / verificação
Interface pública	task_analyzer_spec.md e requirements.yaml	src/task_analyzer.py; imports do Test Harness
Dados e domínios	Seção 3 do contrato e bloco input do YAML	_validar_tarefa e _validar_data
Métricas	Seções 4 e 5 do contrato	analyze_tasks e teste de sucesso
Erro de data	Regra 10 e cenário de erro	TaskValidationError e teste de data inconsistente
Casos de borda	Seções 5 e 6 do contrato	_media, guarda da taxa e dois testes de borda
Qualidade e IA	CONTEXT_RULES.md	Estrutura do código, revisão, PR e CI
Automação	Arquivos de infraestrutura	ci.yml, Dockerfile e docker-compose.yml

Referências internas consultadas

- specs/task_analyzer_spec.md - contrato SDD original
- specs/requirements.yaml - transposição estruturada
- CONTEXT_RULES.md - arquitetura, proibições e governança de IA
- src/task_analyzer.py - implementação vigente
- tests/test_harness.py - cenários automatizados
- .github/workflows/ci.yml - integração contínua
- Dockerfile e docker-compose.yml - execução conteinerizada
- README.md, requirements.txt e .gitignore - operação e organização

Baseline documental

Repositório	Branch	Commit	Data da edição
eduardotrm/sdd-task-analyzer	main	b69a1dcc61150e016d5efb5cecaeac22c702284f	23/10/2026

Fim do documento. Alterações futuras no contrato devem preservar a distinção entre fonte normativa, representação estruturada e documentação descritiva.